

Figure 2: (a) Attention Sparsity in pre-trained LLMs. (b) The distribution of accumulated attention scores with respect to the corresponding word (red scatter) and the co-occurrence times of words in the data (gray curve). The x-axis represents the word index in the vocabulary. (c) The performance comparison between the baseline model with full KV and the model *w.o.* heavy hitter. (d) Comparison between the baseline model with full KV, H₂O with the local statistic, H₂O with the global statistic, and the model with only the most recent KV (Local). Apart from the baseline model, each model is evaluated with 20% KV cache budget.

2.1 Problem Formulation

We formally define the generative process with limited KV cache size. Denote attention query matrix as $Q \in \mathbb{R}^{n \times d}$ and key matrix as $K \in \mathbb{R}^{n \times d}$. $Q_{i,*}$ represents the i -th row of Q and $K_{\leq i,*}$ represents the first i rows of K . Let k denote the budget of space and $k < n$. For simplicity, $K_{S_i,*} (\in \mathbb{R}^{i \times d})$ denotes a sub-matrix of K which selects S_i rows from K . (For the non-selected rows $[i] \setminus S_i$, we put all zeros in that row) Eviction policy is defined as:

Definition 2.1 (Eviction Policy, informal). *Let S_{i-1} denote the source set. Let S_i denote the target set. We defined the eviction policy $g : S_{i-1} \rightarrow S_i$ such that*

- $|S_i| = k$ (KV cache size is not changing over the time)
- $|S_i \setminus S_{i-1}| \leq 1$ or equivalently $|S_i \cap S_{i-1}| \geq k - 1$ (we can evict at most 1 KV in the KV cache)

Then, we define the generative process with our eviction policy.

Definition 2.2 (The generative process with eviction policy, informal). *Let k denote the size of the KV cache. For each $i \in [n]$, for the i -th token, we have*

- Let $S_i \subset [n]$ denote the tokens in KV cache when predicting the i -th token.
- The information we have is a length- i vector $o_i := D_i^{-1} \cdot \exp(Q_{i,*}(K_{S_i,*})^\top)$ (normalized attention)
 - scalar $D_i := (\exp(Q_{i,*}(K_{S_i,*})^\top) - 1_{[i] \setminus S_i}) \cdot \mathbf{1}_i$ (the evicted KV is set to 0, and we need to subtract them when computing the normalization)
 - Replacing S_i by $[i]$ in the above definition of o_i and D_i leads to standard generative process.
- The eviction policy (Definition 2.1) updates S_i based on S_{i-1} and their corresponding information.

Remark 2.3. *Our goal is to find a KV cache eviction policy such that the output of the generative process is similar or comparable to the original one without limiting the cache size.*

3 Observations

We present two key empirical insights of LLMs that inspire the design of H₂O, as follows.

3.1 Sparsity for Small Cache Size

Inspired by previous literature, which reveals the existence of attention sparsity in DistillBERT [35] and bounded-norm self-attention heads [36]. We first show an observation on the sparsity of attention in pre-trained LLMs. Then we discuss how it can potentially unlock the possibility of reducing KV cache size without an accuracy drop. Given the normalized attention score $\text{Softmax}(QK^\top)$ matrix that is calculated by the query matrix Q and the key matrix K , we set the threshold as one percent of the maximum value in each row and calculates the corresponding sparsity.

Observation. We conduct zero-shot inference with the pre-trained OPT model on the validation set of Wiki-Text-103. We plot the layer-wise sparsity within attention blocks and visualize the normalized attention score matrix. The results are presented in Figure 2 (a). We observe that although the LLMs are densely trained, the resulting attention score matrices are highly sparse, with a sparsity over 95% in almost all layers.

Insights. The attention blocks’ sparsity suggests that access to all previous key and value embeddings is unnecessary for generating the next token. This suggests it is possible to evict unessential KV embeddings and reduce the requirement of KV cache during generation.

3.2 Heavy-Hitters for Low Miss Rate

The previous section showed the sparse nature of attention blocks in pre-trained LLMs, which provides the opportunity for designing small KV cache size while still maintaining the performance of LLMs. However, determining the best eviction policy that preserves generation accuracy presents a combinatorial challenge. Although Belady’s Algorithm [37] is optimal and easy to compute for standard cache (offline), it is not applicable for KV cache design. Because once evicting important KVs, it could destroy the performance of LLMs due to the sequential dependency of LLM generation.

Observation. Fortunately, in the early stage of our exploration, we find that the accumulated attention scores of all the tokens within attention blocks follow a power-law distribution, as shown in Figure 2. This suggests the existence of a small set of tokens that are critical during generation. We denote those tokens as heavy-hitters (H_2). In order to verify the importance of these tokens, we compare the quality of LLM generation after masking heavy hitters with that of the original model. Not surprisingly, as shown in Figure 2, the accuracy drops drastically, confirming the importance of those tokens. Additionally, we can see the accumulated attention score of each word (in red dots) have a high correlation with their co-occurrences in the data (gray curve).

Analysis. First, based on H_2 , we see an opportunity to side-step from the combinatorial search problem and design a KV cache eviction policy that preserves the LLM generation quality. We conduct an empirical study implementing a KV cache eviction policy that retains only the H_2 and the recent KV embeddings in the cache. The intuition is that recent words typically exhibit stronger correlations with current tokens. We assess the effectiveness of this eviction policy through pre-trained OPT-30B and six downstream tasks. The outcomes of these evaluations are illustrated in Figure 2. It is obvious that the H_2 based eviction policy can largely reduce the KV cache size without degrading the performance of OPT-30B.

Moreover, during the post analysis, inspired by [38], we find that H_2 based policy is related to the classical greedy algorithm (a polynomial-time algorithm with provable guarantees) under the assumption that the attention schema is submodular. We present details in Appendix D.

Lemma 3.1 (informal). *Assuming the attention scheme is submodular, then greedily constructing the set S_i (without cache size limitation) satisfies the near-optimal property in terms of submodular.*

4 Heavy-Hitter Oracle

The goal of this section is to propose the greedy algorithm using the H_2 -based policy and to show the provable guarantees. We first present the H_2 -based policy called H_2O cache eviction policy and formulate its deployment in LLM generation as a variant of submodular maximization problem, named *dynamic submodular*. Then we present H_2O in the generative process, followed by a practical example of deploying our proposal. Finally, we provide theoretical guarantees for H_2O and show our efficient system implementation.

4.1 Greedy Algorithm for Low-Cost Policy

We have shown a simple yet effective KV cache policy based on H_2 . However, it is impractical to deploy such an algorithm because we do not have access to the future-generated tokens. Fortunately, we empirically observe that local H_2 , which is calculated using local statistics at every decoding step by summing up the attention scores of the previous tokens, is equally effective as taking into account the attention of future tokens (Figure 2). In the following, we formally define this dynamic attention score computation (with space limitation) as a novel dynamic submodular type problem.

Definition 4.1 (Dynamic submodular framework, informal). *Define function $F : 2^{[n]} \times 2^{[n]} \rightarrow \mathbb{R}$, then for any set $Z \subset [n]$, we assume that $F(Z, \cdot) : 2^{[n]} \rightarrow \mathbb{R}$ is a submodular function w.r.t. to Z , i.e.,*

- For all sets $X, Y \subset [n]$ satisfy that $Z \subset X \subset Y$,
- For all element $x \in [n]$ satisfy that $x \in [n] \setminus Y$,

we have $f(X \cup \{x\}) - f(X) \geq f(Y \cup \{x\}) - f(Y)$, where $f(\cdot) := F(Z, \cdot)$.

Remark 4.2. We provide practical insights of Definition 4.1. X denotes the existing words in the KV cache. Y is any superset of X . x can be viewed as a “word” which is either newly added to KV cache or existing deleted from KV cache. An example f can be attention score, i.e., see Algorithm 1.

If we load the sequence of $S_1, S_2, \dots, S_n$ (we promise that $|S_i| \leq k$ and $|S_i \setminus S_{i-1}| \leq 1$) into Definition 4.1, i.e., for each $i \in [n]$, we choose $Z = S_i$, then it becomes a particular instance of the dynamic submodular problem.

Next, we provide a formal description of our algorithm, followed by an example.

Definition 4.3 (H₂O Eviction Policy). Let $F_{\text{score}} : 2^{[n]} \rightarrow \mathbb{R}$ denote certain score function. Let S_{i-1} denote the source set. Let S_i denote the target set. We defined the eviction policy $g : S_{i-1} \rightarrow S_i$ s.t.

- $|S_i| = k$ (KV cache size is not changing over the time)
- $|S_i \setminus S_{i-1}| \leq 1$ or equivalently $|S_i \cap S_{i-1}| \geq k - 1$ (we can evict at most 1 KV in the KV cache)
- We construct $S_i \leftarrow (S_{i-1} \cup \{i\}) \setminus \{u\}$ as $u \leftarrow \arg \max_{v \in (S_{i-1} \cup \{i\})} F_{\text{score}}(S_{i-1} \cup \{i\} \setminus \{v\})$

To describe our algorithm (Algorithm 1), we choose a particular instantiation of the function F_{score} , i.e., the summation of that sets in the attention matrix.

Algorithm 1 H₂ Eviction Algorithm

```

1: procedure H2_EVICTON( $Q, K \in \mathbb{R}^{n \times d}, k \in \mathbb{N}$ )
2:   Let  $k$  denote the budget size of cache
3:    $S_0 \leftarrow \emptyset$ 
4:   for  $i = 1 \rightarrow n$  do
5:     if  $i \leq k$  then
6:        $S_i \leftarrow S_{i-1} \cup \{i\}$ 
7:     else
8:        $D_i \leftarrow (\exp(Q_{i,*}(K_{S_{i-1},*})^\top) - 1_{[i] \setminus S_{i-1}}) \cdot \mathbf{1}_i$ 
9:        $o_i \leftarrow D_i^{-1} \cdot (\exp(Q_{i,*}(K_{S_{i-1},*})^\top) - 1_{[i] \setminus S_{i-1}})$ 
10:       $F_{\text{score}}(T) := \sum_{s \in T} o_s$ 
11:       $G_i \leftarrow S_{i-1} \cup \{i\}$ 
12:       $u \leftarrow \arg \max_{v \in G_i} F_{\text{score}}(S_{i-1} \cup \{i\} \setminus \{v\})$ 
13:       $S_i \leftarrow (S_{i-1} \cup \{i\}) \setminus \{u\}$ 
14:    end if
15:  end for
16: end procedure

```

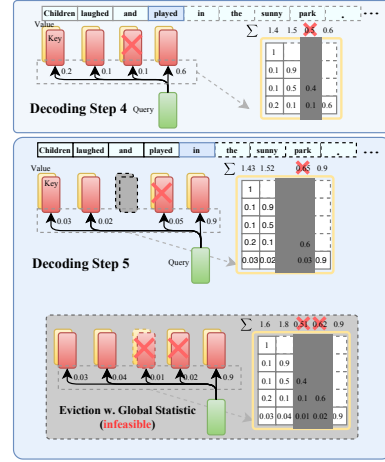


Figure 3: Illustration of Algorithm 1 during two consecutive decoding steps.

Figure 3 presents an illustrative example of our H₂ Eviction Algorithm. We assume that the budget size of KV cache is 3. Following the completion of the fourth decoding step, the KV embeddings associated with the third token are evicted based on the accumulated attention score. Consequently, these evicted KV embeddings become inaccessible in the subsequent decoding steps.

4.2 Theoretical Guarantee and System Implementation

We state a theoretical result as follows. The proofs and more details are provided in Appendix D.

Theorem 4.4 (informal). Under the mild assumption, let k denote the budget of space limitation. If for each token, we greedily compute the attention score based on top- k choice, then we can show the set $\tilde{S}_i$ we generate each for token i satisfy that $f(\tilde{S}_i) \geq (1 - \alpha)(1 - 1/e) \max_{|S|=k} f(S) - \beta$, where $\alpha, \beta > 0$ are parameters.

Remark 4.5. We remark the above theorem provides a theoretical explanation of why can we hope our greedy algorithm (with cache limitation) can provide a good solution to the problem.

Implementation Details. We provide a general framework that can support any KV cache eviction algorithm and enhance throughput and reduce the latency of LLM generation with careful implementation. For example, to ensure I/O efficiency, we do not swap memory when stored KV is evicted, but directly fill with newly-added KV. More details are included in Appendix A.